

Personal Project System Design
Document
Minor AI for Society

Casey Stijlaart

Table of Contents

1. Document history	1
2. Introduction	2
3. System Description	3
3.1. Extensions	3
4. System Design	5
4.1. Components	5
4.2. Use Cases	7
4.3. Requirements	9
4.4. Requirements Matrix	11
5. System behavior	12
5.1. Sequence Diagrams	12
5.2. Class Diagram : ESP Firmware	16
6. Hardware Diagram	17

1. Document history

Table 1. Document History

Version	Date	Status	Author	Description	Remarks
0.1	30-03-2026	Draft	Casey Stijlaart	Initial Draft	-
0.1	01-04-2026	Draft	Casey Stijlaart	Additional Details and design	-
1.0	08-04-2026	First Version	Casey Stijlaart	Completed	-

2. Introduction

This document serves as a comprehensive guide detailing the design and architecture of the embedded system that focusses on plant monitoring and care. The system is designed to monitor various environmental parameters such as soil moisture, temperature, humidity, and light intensity to ensure optimal conditions for plant health. It contains a detailed system description with multiple diagrams defining all of the hardware components and the behavior of the system.

3. System Description

The plant health monitoring system is an embedded solution designed to monitor environmental conditions and provide feedback on plant health. The system is centered around an ESP32 microcontroller, which collects data from sensors such as soil moisture, temperature, and humidity.

The ESP32 processes this data locally using a lightweight machine learning model to determine the plant's health status (e.g., healthy, moderate stress, high stress). The results are presented to the user through a simple interface, allowing monitoring and basic interaction, such as setting plant-specific preferences.

The system follows a modular architecture consisting of sensors (data collection), the ESP32 (processing), and a user interface (visualization). It is designed to operate efficiently on edge hardware with minimal external dependencies, while remaining extensible for future improvements.

3.1. Extensions

The following extensions are categorized using the MoSCoW prioritization method to define which features are essential, desirable, optional, or out of scope for the current version of the plant health monitoring system.

3.1.1. Must

These are the core extensions or capabilities required for the system to meet its primary objective.

- The system must monitor key environmental factors relevant to plant health, such as soil moisture, temperature, and humidity.
- The system must process sensor data and provide plant health feedback to the user.
- The system must present the measured values and plant status through a user interface.
- The system must operate on the ESP32 platform as the main embedded controller.
- The system must support a basic warning or alert when plant conditions fall outside acceptable ranges.
- The system should provide clearer interpretation of plant condition, for example by classifying status as healthy, moderate stress, or high stress.

3.1.2. Should

These features are important and would significantly improve the system, but the system can still function without them in the first version.

- The system should allow the user to enter plant-specific preferences, such as whether a plant prefers low, medium, or high moisture.
- The system should store historical sensor readings for later review.
- The system could support multiple plants instead of only one monitored plant.

3.1.3. Could

These are optional enhancements that may improve usability or functionality if time and resources allow.

- The system could include additional sensors, such as light intensity or air quality sensors.
- The system could provide trend visualization, such as graphs of moisture or temperature over time.
- The system could include personalized recommendations for plant care actions.

3.1.4. Won't

These items are intentionally excluded from the current scope of the project.

- The system won't include full automation of plant care, such as automatic watering, in the current version.
- The system won't use a complex cloud backend as part of the initial prototype.
- The system won't support large-scale commercial greenhouse management.
- The system won't include advanced computer vision or camera-based plant disease detection.
- The system won't aim to support every plant species with fully customized biological models.

4. System Design

4.1. Components

The system is composed of several components that work together to monitor plant health and provide feedback to the user. To outline each component and their role, an overview is given, see [Table 2](#). Next to that, a component diagram, [Figure 1](#) is given to illustrate the relationships between the components.

Table 2. Component Description

Component	Description
Edge Node (EN)	The Edge Node is responsible for receiving sensor data, performing preprocessing and local ML inference, and managing device-specific configuration. It communicates with the central storage to upload measurements and receives configuration during provisioning.
Sensor Array (SA)	The sensor array consists of various sensors that monitor the plant's environment, such as soil moisture, temperature, humidity, and light levels. These sensors provide real-time data to the Edge Node.
User Interface (UI)	The user interface allows the user to manage plant profiles, view plant health status and historical data, and configure devices. It communicates with the central storage for data access and uses BLE to provision nearby Edge Nodes.
Data Storage (DS)	Stores plant profiles, user-defined settings, and historical sensor data. Acts as the central source of truth for the system and enables remote access to plant data.

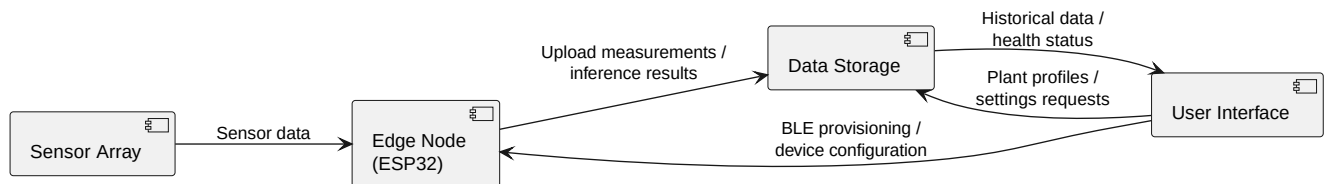


Figure 1. Component Diagram

4.1.1. Messages Between Components

To showcase how each component is connected to another, the overall message that connects them are defined, see [\[Messages between components\]](#).

Table 3. Messages between components

Component	Message name	From	To	Description
Sensor Array	GetData()	Edge Node	Sensor Array	Retrieves measured sensor data from the sensor array.
Edge Node	Send(data)	Edge Node	Data Storage	Sends sensor readings and inferred plant health data to the central storage.
User Interface	AddPlant(plant_info)	Interface	Data Storage	Stores a new plant profile with user-defined settings such as soil moisture and light preferences.
User Interface	GetPlantData(plant_id)	Interface	Data Storage	Requests historical and current plant data for visualization.
User Interface	DisplayHealthStatus(health_status)	Data Storage	Interface	Displays plant health status retrieved from stored data.
User Interface	ProvisionDevice(settings)	Interface	Edge Node (BLE)	Sends plant configuration to a nearby available Edge Node via BLE during setup.
Edge Node	StoreLocalConfig(settings)	Edge Node	Edge Node	Stores received configuration locally for autonomous operation.
User Interface	GetAvailableDevices()	Interface	Edge Node (BLE)	Scans for nearby unassigned Edge Nodes available for provisioning.

4.2. Use Cases

This section describes the main interactions between the user and the system. The use cases focus on plant monitoring, configuration, and data visualization.

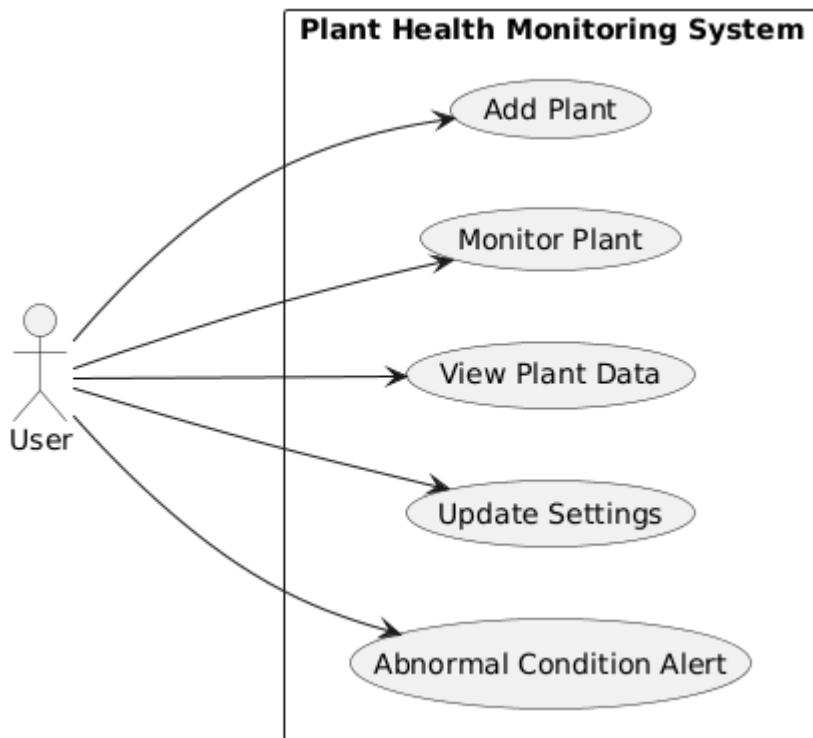


Figure 2. Use Case Diagram

[\[Use Case Diagram\]](#) depicts some of the use cases. It shows system boundary and specifies how actors interact with the system. Each use case will be described in the following table.

Table 4. Use Case 01

ID: UC01	Use Case Name: Add Plant
Description	The user adds a new plant by providing plant details such as name, type, and care requirements. The system stores this information for monitoring.
Actor(s)	User

Table 5. Use Case 02

ID: UC02	Use Case Name: Monitor Plant
Description	The system monitors plant conditions using sensor data and ML inference, and makes the results available to the user.
Actor(s)	User

Table 6. Use Case 03

ID: UC03	Use Case Name: View Plant Data
Description	The user views current and historical plant data through the system interface.
Actor(s)	User

Table 7. Use Case 04

ID: UC04	Use Case Name: Update Settings
Description	The user updates plant care settings such as soil moisture thresholds. The system stores and applies the updated configuration.
Actor(s)	User

Table 8. Use Case 05

ID: UC05	Use Case Name: Abnormal Condition Alert
Description	The system detects abnormal plant conditions and notifies the user with relevant information and suggested actions.
Actor(s)	User

4.3. Requirements

A system has various requirements, each one covering an important aspect of how the system must work or behavior. These requirements are divided into the user-, functional- and non-functional requirements.

4.3.1. User Requirements

Table 9. User Requirements

ID	Requirement	Use Case(s)
UR_001	The user should be able to add a new plant with its care requirements.	UC01
UR_002	The user should be able to monitor the health status of a plant.	UC02, UC03
UR_003	The user should be able to view current and historical plant data.	UC03
UR_004	The user should be able to update plant care settings.	UC04
UR_005	The user should receive notifications when abnormal conditions occur.	UC05

4.3.2. Functional Requirements

Table 10. Functional Requirements

ID	Requirement	Use Case(s)
FR_001	The user interface must allow the user to input plant details when adding a plant.	UC01
FR_002	The data storage must store plant profiles and associated care settings.	UC01
FR_003	The sensor array must provide environmental data such as soil moisture, temperature, and humidity.	UC02
FR_004	The edge node must read and preprocess sensor data.	UC02
FR_005	The edge node must use an ML model to predict plant health.	UC02

ID	Requirement	Use Case(s)
FR_006	The edge node must send sensor data and predictions to the data storage.	UC02
FR_007	The data storage must provide plant data when requested by the user interface.	UC03
FR_008	The user interface must display current and historical plant data.	UC03
FR_009	The user interface must allow the user to update plant care settings.	UC04
FR_010	The data storage must update and persist modified plant settings.	UC04
FR_011	The edge node must detect abnormal conditions based on sensor data and predictions.	UC05
FR_012	The user interface must notify the user when abnormal conditions are detected.	UC05

4.3.3. Non-Functional Requirements

Table 11. Non-Functional Requirements

ID	Requirement
NFR_001	The system should be modular so components can be updated or replaced independently.
NFR_002	The system should support multiple edge nodes and plants.
NFR_003	The edge node must operate within embedded hardware constraints such as limited memory and processing power.
NFR_004	The system should allow security and privacy features to be added in future iterations.

4.4. Requirements Matrix

The requirements matrix provides a consolidated view of all the requirements, showing how they relate to each other and to the use cases. This matrix helps ensure that all requirements are covered and that there are no gaps in the system design.

Table 12. Use Cases vs User Requirements Coverage

	UR_001	UR_002	UR_003	UR_004	UR_005
UC01	X				
UC02		X			
UC03		X	X		
UC04				X	
UC05					X

Table 13. Functional Requirements vs User Requirements Coverage

	UR_001	UR_002	UR_003	UR_004	UR_005
FR_001	X				
FR_002	X				
FR_003		X			
FR_004		X			
FR_005		X			
FR_006		X			
FR_007			X		
FR_008			X		
FR_009				X	
FR_010				X	
FR_011					X
FR_012					X

5. System behavior

5.1. Sequence Diagrams

Per use case, there is an associated sequence diagram that describes the interactions between the system components to achieve the desired functionality. These diagrams provide a visual representation of the flow of messages and the order of operations within the system.

5.1.1. UC01 : Add Plant

Scenario: A user is able to add a new plant to the system by entering its details through the user interface.

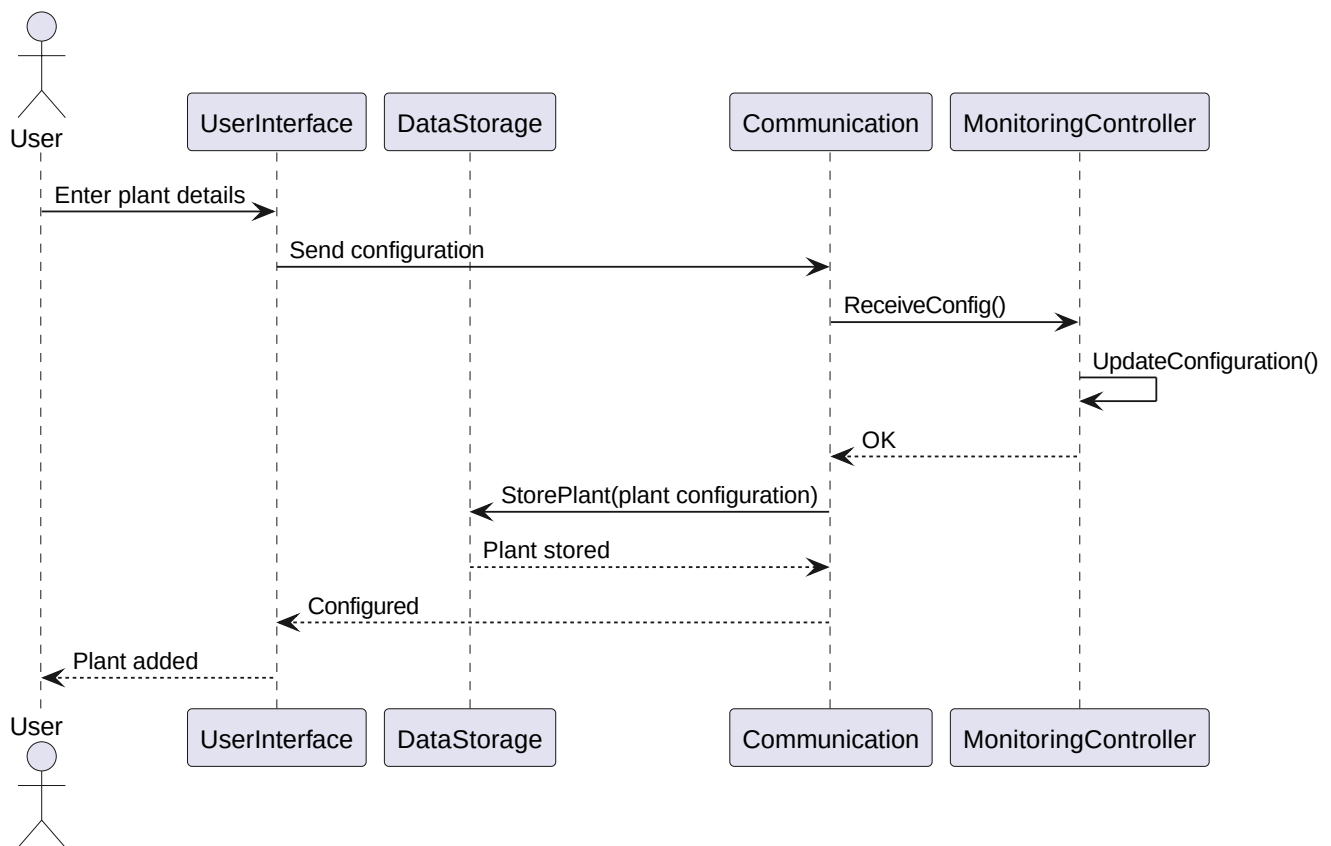


Figure 3. UC01 Sequence Diagram

Scenario: If there is an error during the plant addition process, the system should handle it gracefully and inform the user.

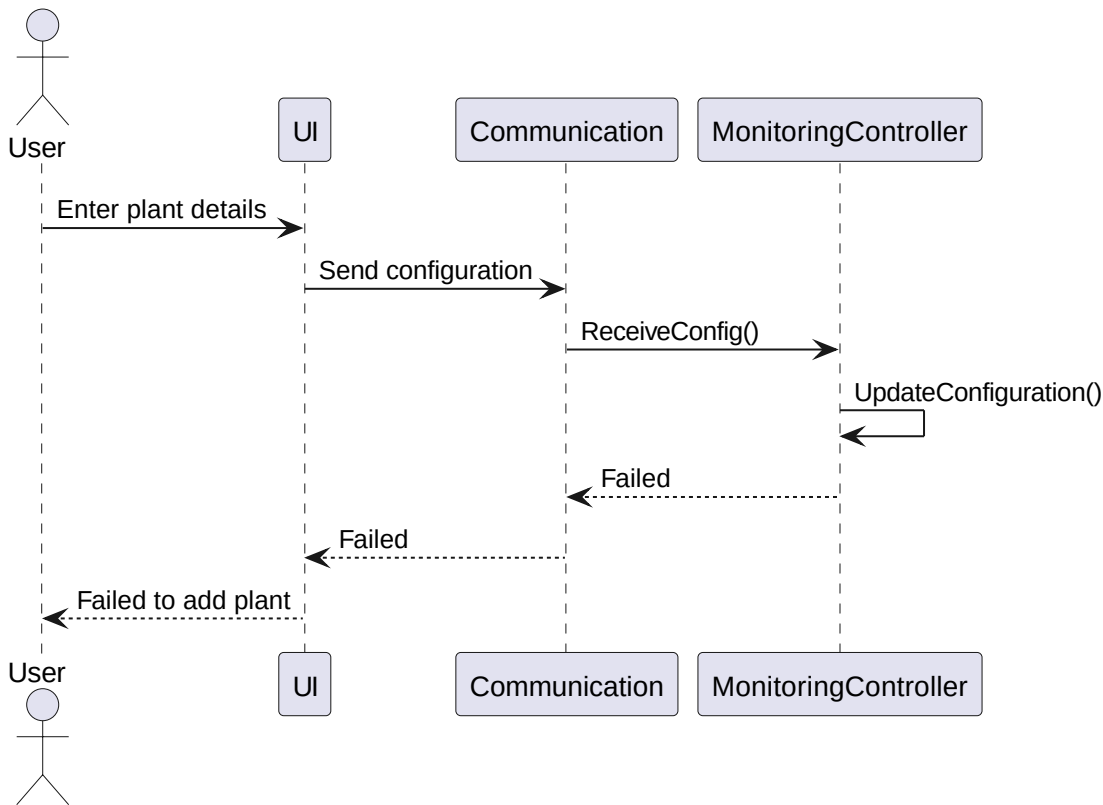


Figure 4. UC01 Error Handling Sequence Diagram

5.1.2. UC02 + UC05 : Monitor Plant + Abnormal Condition Alert

Scenario: The system continuously monitors the plant's conditions and sends alerts to the user if any abnormal conditions are detected.

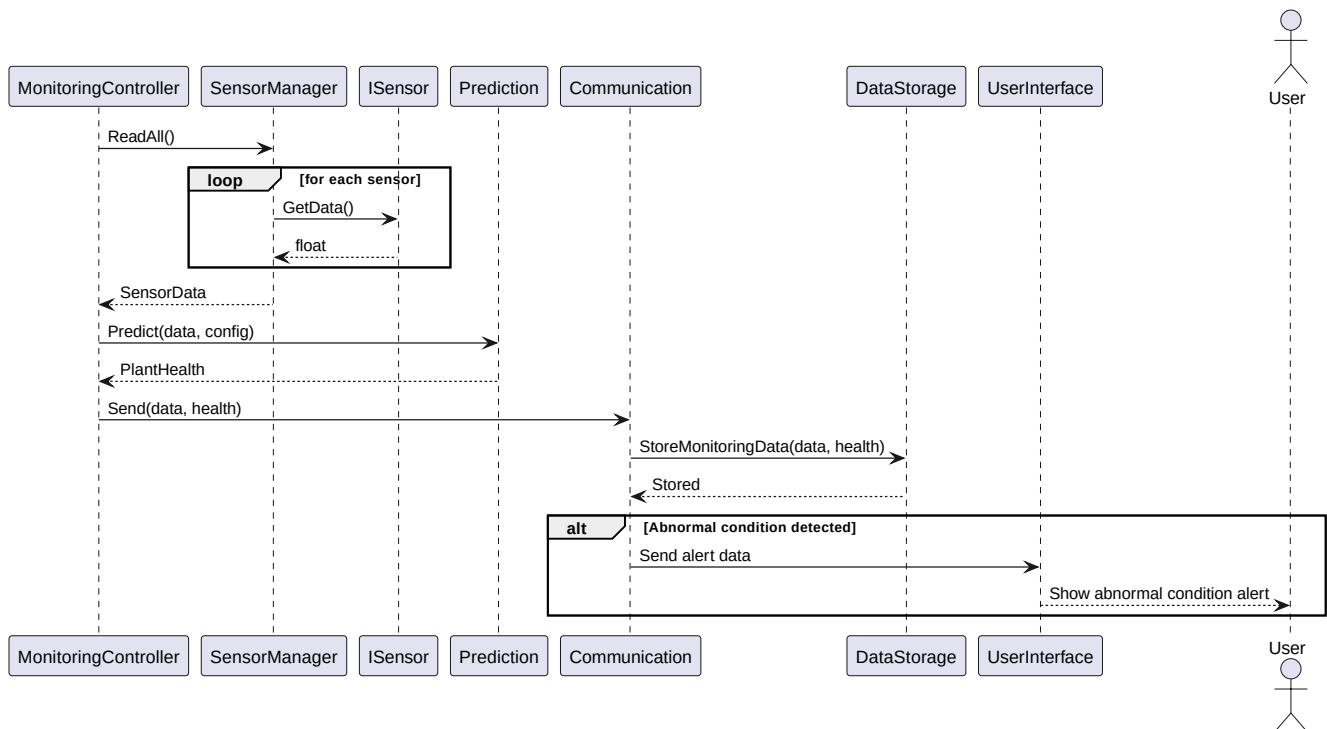


Figure 5. UC02 + UC05 Sequence Diagram

Scenario : The system is not able to read sensor data or predict plant health, and it should handle these errors appropriately.

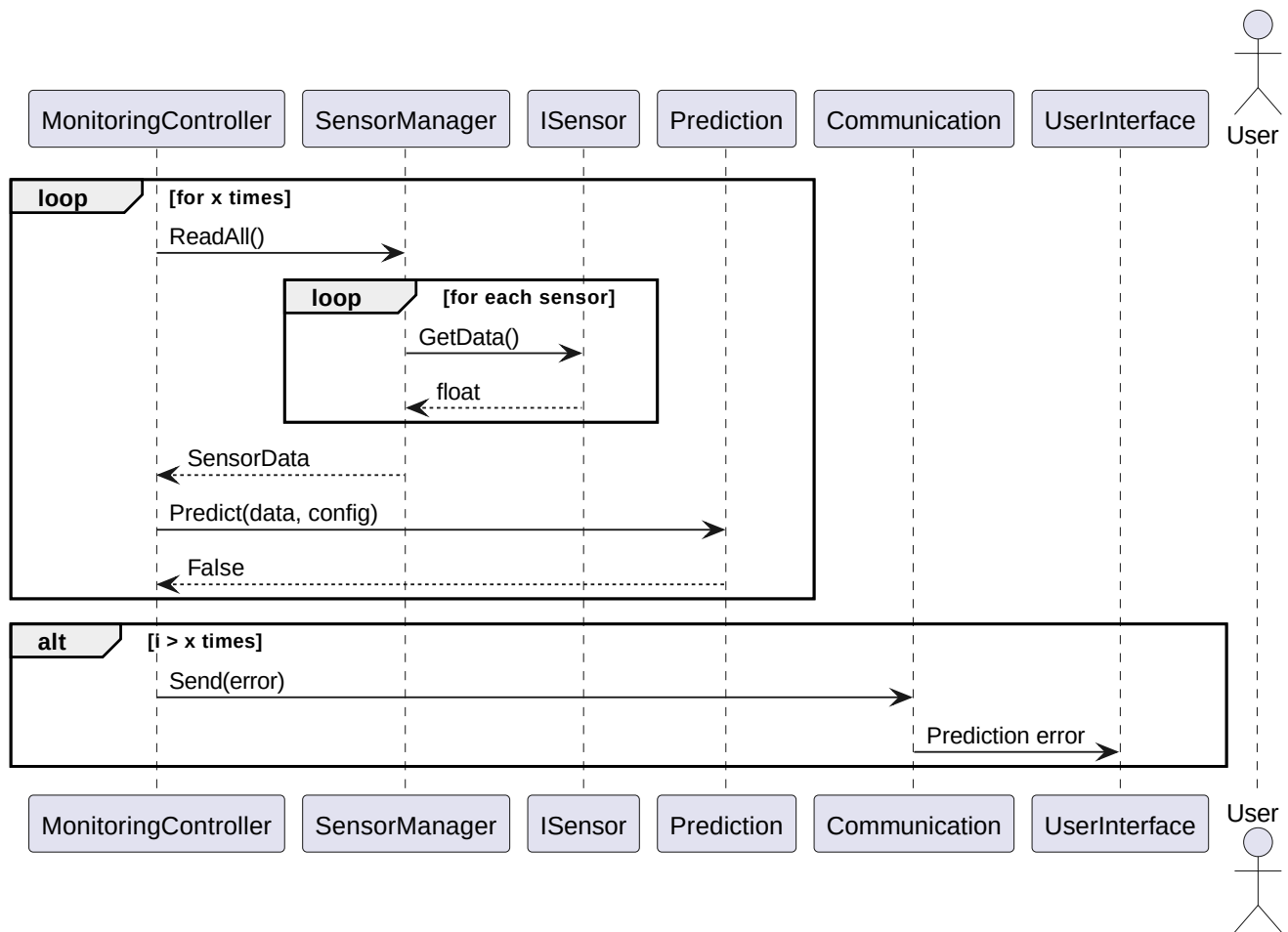


Figure 6. UC02 + UC05 Error Handling Sequence Diagram

5.1.3. UC03 + UC04 : View Plant Data + Update Settings

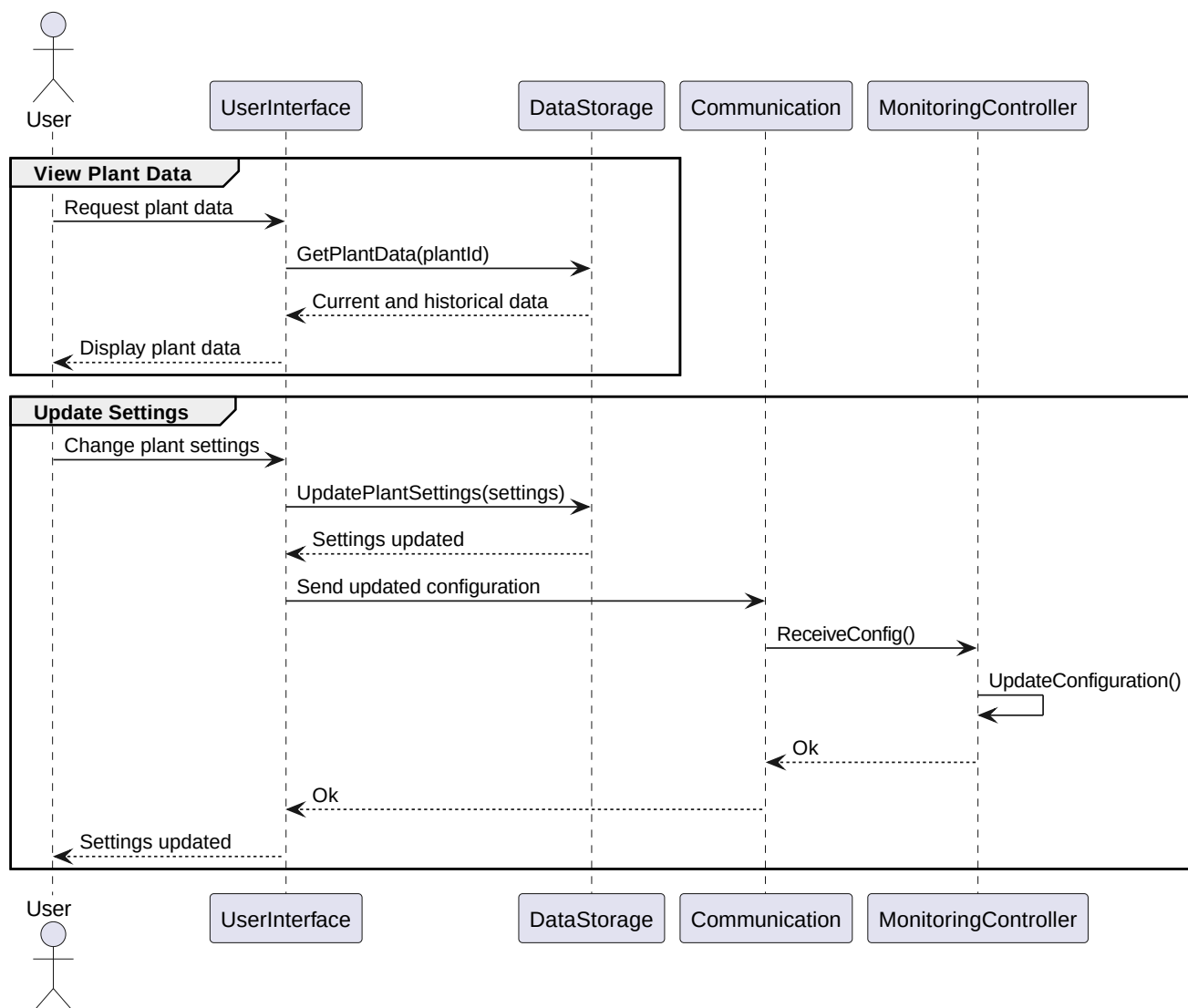


Figure 7. UC03 + UC04 Sequence Diagram

5.2. Class Diagram : ESP Firmware

The class diagram for the ESP firmware is designed to illustrate the structure and relationships between the various classes that make up the firmware. The diagram includes classes for handling sensor data, communication protocols, and device management.

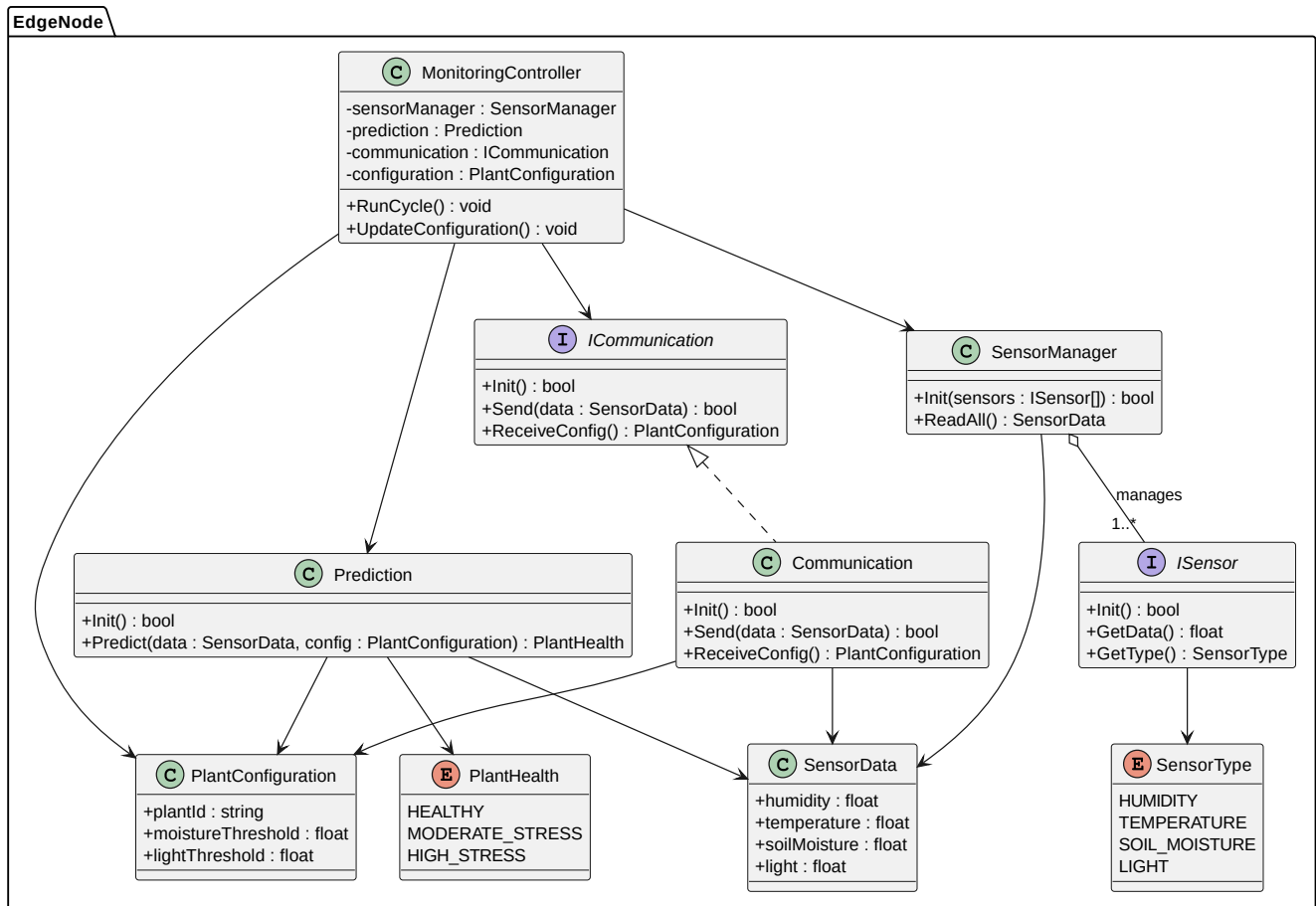


Figure 8. Class Diagram

6. Hardware Diagram

The system will be composed of several hardware modules. Some logical units run on multiple physical blocks. The table, [\[Used hardware components\]](#), describes the individual hardware components. The diagram, [\[Hardware connection\]](#), the connection between the various hardware components.

Table 14. Used hardware components

Hardware block	Description
ESP32	The ESP32 will be used as the ML unit. It will run the machine learning models and will be responsible for processing the data from the various sensors and making decisions and predictions based upon that data.
Environmental Sensors	There are various sensors used to measure data with the help of the ESP32 CP2102.
Interface (IF)	The interface is used to display the data and predictions from the ESP32 CP2102. It will also be used to receive user input for certain actions.

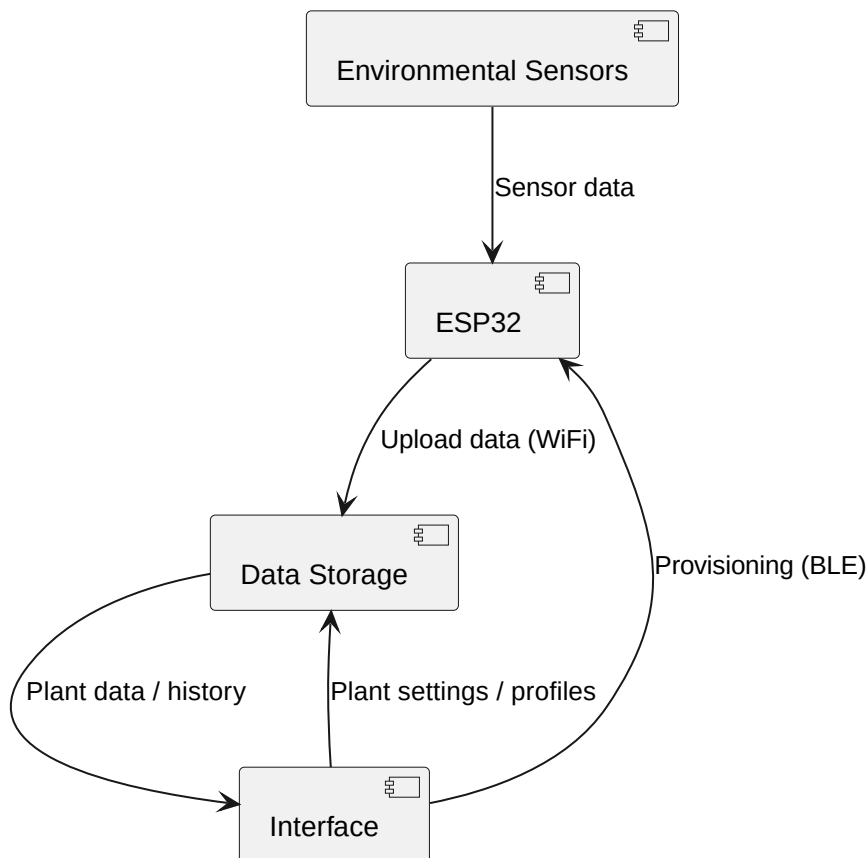


Figure 9. Hardware connection